

Critérios de Seleção de Repositórios

Projeto: análise estática de qualidade e segurança de código Java (ISO/IEC 25010) com predição por Machine Learning.

O corpus atual usa 10 repositórios da `NationalSecurityAgency`, mas isso não é uma exigência do método. Qualquer repositório público do GitHub que atenda aos 10 critérios abaixo pode ser analisado pelo pipeline, independente de organização ou domínio.

Os 10 critérios

#	CRITÉRIO	LIMIAR	POR QUE É OBRIGATÓRIO
1	Java dominante	$\geq 70\%$ do LOC	Todas as ferramentas (CK, CodeQL, PyDriller, lizard) são Java-only. Fora disso, o dataset sai vazio.
2	Público no GitHub, <code>owner/repo</code>	clone anônimo	Sem LFS ou submódulos, e o nome não pode conter <code>--</code> (usado como separador interno de pastas).
3	Histórico Git rico	≥ 300 commits, ≥ 5 autores, ≥ 2 anos	7 features de ML vêm do histórico. Histórico <i>squashed</i> torna todas constantes.
4	Analisável sem build	zero compilação	O estudo é 100% estático. Código gerado em build (Protobuf, ANTLR, Lombok) não existe no clone.
5	Manifesto Maven/Gradle	versões literais	A análise de dependências (OSV) exige versão exata. Ant, Bazel e <i>version catalog</i> não são reconhecidos.
6	Volume adequado	200–5.000 arquivos <code>.java</code>	Abaixo: sem dados para treinar. Acima: estoura os <i>timeouts</i> das ferramentas (o <code>ghidra</code> , com 2M LOC, é o caso-limite).
7	Superfície de segurança real	1%–30% de arquivos com achado	O rótulo vem de SAST (Semgrep/CodeQL). Código sem rede/cripto/persistência gera zero positivos e inviabiliza o treinamento.
8	Código OO com variância	classes, herança, métodos reais	As métricas CK (CBO, LCOM, DIT) precisam de variação. Só DTOs ou exemplos triviais não geram sinal.
9	Higiene de encoding/caminhos	UTF-8, caminhos < 260 chars	Restrição real de execução em Windows; <i>symlinks</i> e árvores duplicadas quebram a junção das métricas.
10	Corpus diverso	≥ 5 repos, 1 por organização	A validação é <code>GroupKFold</code> agrupado por repositório: cada <i>fold</i> testa em um repositório nunca visto.

Três pontos de atenção metodológica

1. O critério 7 é o mais decisivo. O alvo de classificação (`has_security_risk`) é definido por achados de SAST. Um repositório sem superfície de segurança — biblioteca matemática, algoritmos, estruturas de dados — produz apenas rótulos negativos, e o *fold* correspondente é automaticamente descartado pelo pipeline.

2. O critério 10 corrige uma limitação do corpus atual. Dos 10 repositórios analisados, 4 pertencem ao mesmo ecossistema (DataWave) — mesma organização, mesmo estilo de código. Isso é uma variável de confusão: o modelo tende a aprender o padrão de uma organização em vez das características gerais de código inseguro. A regra “um repositório por organização” no novo corpus elimina essa ameaça à validade externa.

3. Os critérios 4, 5 e 6 são consequência do desenho sem build. Analisar apenas o código-fonte (sem compilar) é o que torna o estudo reproduzível e escalável, mas impõe limites conhecidos: dependências transitivas não são resolvidas e código gerado não é medido. Ambos já constam como limitações declaradas do trabalho.

Candidatos sugeridos para o novo corpus

Públicos no GitHub, Java, build Maven, organizações distintas e alta relevância na comunidade técnica:

REPOSITÓRIO	DOMÍNIO
<code>netty/netty</code>	I/O de rede e TLS
<code>apache/shiro</code>	autenticação e autorização
<code>alibaba/nacos</code>	<i>service registry</i> e configuração
<code>apache/rocketmq</code>	mensageria distribuída
<code>apache/zookeeper</code>	coordenação distribuída
<code>apache/dubbo</code>	RPC e serialização
<code>jenkinsci/jenkins</code>	servidor de integração contínua
<code>apache/commons-compress</code>	parsing de arquivos compactados
<code>thingsboard/thingsboard</code>	plataforma IoT
<code>google/guava</code>	biblioteca utilitária (caso de contraste)

Descartados apesar da popularidade: `elasticsearch`, `hadoop` e `flink` (volume excessivo — critério 6); `spring-boot` (formato de dependências incompatível — critério 5); `tomcat` (build Ant — critério 5); `okhttp` (migrado para Kotlin — critério 1).